Laravel Complete Learning Guide & Curriculum

A structured path from **absolute beginner** to **production-ready Laravel developer**, with hands-on notes and the **TaskForge** example project.

Your environment: PHP 8.2 + Laravel 12 (TaskForge). Laravel 13 requires PHP 8.3+.

How to Use This Guide

1. **Read modules in order** - each builds on the previous (`docs/00` -> `docs/18`).
2. **Code along in TaskForge** - run commands and edit files as each module instructs.
3. **Complete checkpoints** - every module ends with exercises and a self-check quiz.
4. **Track progress** - use the checklist below.

Curriculum Roadmap (~ 16-24 weeks at 10-15 hrs/week)

Phase	Weeks	Modules	Goal
Foundation	1-3	00-02	PHP, tooling, Laravel anatomy
Core Web	4-7	03-06	Routes, views, DB, auth
Intermediate	8-11	07-11	APIs, assets, validation, storage, queues
Advanced	12-14	12-15	Events, cache, testing, security
Production	15-18	16-18	Deploy, monitor, maintain, scale

Progress Checklist

Phase 1 - Foundation

- [ ] Module 00: Environment & first Laravel app
- [ ] Module 01: PHP essentials for Laravel
- [ ] Module 02: Request lifecycle & project structure

Phase 2 - Core Web Development

- [ ] Module 03: Routing & controllers
- [ ] Module 04: Blade templates & layouts
- [ ] Module 05: Database, migrations & Eloquent
- [ ] Module 06: Authentication & authorization

Phase 3 - Intermediate Skills

- [ ] Module 07: REST API development
- [ ] Module 08: Frontend assets (Vite, Tailwind)
- [ ] Module 09: Validation, forms & requests
- [ ] Module 10: File storage & media
- [ ] Module 11: Queues, jobs & scheduling

Phase 4 - Advanced & Production

- [ ] Module 12: Events, listeners & notifications
- [ ] Module 13: Caching & performance
- [ ] Module 14: Testing (unit, feature, browser)
- [ ] Module 15: Security hardening
- [ ] Module 16: Deployment & hosting
- [ ] Module 17: Maintenance & monitoring
- [ ] Module 18: Advanced patterns & architecture

Repository Structure

```
laravel/
|-- README.md                 <- You are here (curriculum overview)
|-- docs/                     <- Detailed learning modules (00-18)
|   |-- 00-getting-started.md
|   |-- 01-php-fundamentals.md
|   `-- ...
`-- taskforge/                <- Example project (task management app)
|-- README.md             <- Project-specific setup & feature map
|-- app/                  <- Application code (models, controllers, etc.)
|-- database/             <- Migrations, seeders, factories
|-- resources/views/      <- Blade templates
|-- routes/               <- Web & API routes
|-- tests/                <- PHPUnit tests
`-- deploy/               <- Deployment configs & scripts
```

TaskForge - Example Project

TaskForge is a team task management application that demonstrates every major Laravel concept:

Feature	Laravel Concepts
User registration & login	Breeze, middleware, policies
Projects & tasks CRUD	Resource controllers, Eloquent relationships
Task assignments & comments	Many-to-many, polymorphic relations
Due-date reminders	Queued jobs, mail, scheduling
Activity log	Events, listeners, observers
REST API	API resources, Sanctum tokens
File attachments	Storage, validation
Dashboard stats	Query scopes, caching
Full test suite	Feature & unit tests

Quick Start

```
cd taskforge
cp .env.example .env
php artisan key:generate
php artisan migrate --seed
php artisan serve
```

Visit `http://localhost:8000` - login with `admin@taskforge.test` / `password`.

See taskforge/README.md for the full feature map and module cross-references.

Recommended Learning Schedule

Daily (1-2 hours)

- 30 min reading the module
- 45 min coding in TaskForge
- 15 min review & notes

Weekly

- Complete 1 module
- Run `php artisan test` after each module
- Push a git commit summarizing what you learned

Monthly Milestones

Month	Milestone
1	TaskForge CRUD + auth working locally
2	API endpoints + validation + file uploads
3	Queues, events, 80%+ test coverage
4	Deployed to staging with CI/CD

Essential Resources

Resource	URL
Official Laravel Docs	https://laravel.com/docs
Laracasts (video)	https://laracasts.com
Laravel News	https://laravel-news.com
PHP The Right Way	https://phptherightway.com

Next Step

Open **docs/00-getting-started.md** and begin Module 00.

Module 00: Getting Started with Laravel

Duration: 3-5 days | **Level:** Beginner | **Prerequisites:** Basic computer skills

Learning Objectives

By the end of this module you will:

- Understand what Laravel is and when to use it
- Install PHP, Composer, and a local development environment
- Create your first Laravel project
- Run the development server and Artisan commands
- Navigate the project folder structure

1. What Is Laravel?

Laravel is a **PHP web application framework** that provides:

Feature	Benefit
MVC architecture	Separates logic, data, and presentation
Eloquent ORM	Database work without raw SQL
Blade templating	Clean, reusable HTML views
Built-in auth, queues, mail	Production features out of the box
Massive ecosystem	Packages for payments, search, admin panels

Laravel follows the **convention over configuration** philosophy - sensible defaults so you write less boilerplate.

2. Prerequisites

Required Software

Tool	Minimum Version	Purpose
PHP	8.2+	Runtime (you have 8.2.12 [x])
Composer	2.x	PHP dependency manager
Node.js	18+	Frontend asset compilation (Vite)
Git	Any recent	Version control
Database	SQLite/MySQL/PostgreSQL	Data storage

Windows Setup (XAMPP - your current setup)

You already have PHP via XAMPP at `C:\xampp\php\php.exe`. Ensure these PHP extensions are enabled in `php.ini`:

```
extension=curl
extension=fileinfo
extension=mbstring
extension=openssl
extension=pdo_mysql
extension=pdo_sqlite
extension=tokenizer
extension=xml
```

Verify:

```
php -v
composer --version
php -m | findstr pdo
```

Optional but Recommended

- **Laragon** or **Herd** - simpler Laravel dev environments on Windows
- **VS Code** or **PhpStorm** with PHP Intelephense
- **TablePlus** or **DBeaver** - database GUI

3. Creating a Laravel Project

```
composer create-project laravel/laravel my-app
cd my-app
cp .env.example .env        # Windows: copy .env.example .env
php artisan key:generate
```

What Composer Does

1. Downloads `laravel/laravel` (the application skeleton)
2. Resolves all dependencies into `vendor/`
3. Generates `composer.lock` for reproducible installs

Note: Laravel 13 requires PHP 8.3+. With PHP 8.2 you get Laravel 12 - fully capable for learning and production.

---

4. Environment Configuration (`.env`)

The `.env` file holds secrets and environment-specific settings:

```
APP_NAME=TaskForge
APP_ENV=local
APP_KEY=base64:...          # Generated by key:generate
APP_DEBUG=true
APP_URL=http://localhost

DB_CONNECTION=sqlite        # Easiest for learning
# DB_CONNECTION=mysql
# DB_HOST=127.0.0.1
# DB_PORT=3306
# DB_DATABASE=taskforge
# DB_USERNAME=root
# DB_PASSWORD=

QUEUE_CONNECTION=database
CACHE_STORE=database
SESSION_DRIVER=database
```

SQLite Quick Setup (TaskForge default)

```
touch database/database.sqlite   # Windows: type nul > database\database.sqlite
php artisan migrate
```

---

5. Running the Application

```
# Start development server
php artisan serve
# -> http://127.0.0.1:8000

# Watch logs in real time (Laravel 11+)
php artisan pail

# Compile frontend assets (separate terminal)
npm install && npm run dev
```

Health Check

Laravel 11+ registers `/up` automatically. Visit `http://localhost:8000/up` - should return `{"status":"ok"}`.

---

6. Artisan - The Laravel CLI

Artisan is Laravel's command-line interface:

```
php artisan list                    # All commands
php artisan make:model Task -mfc    # Model + migration + factory + controller
php artisan migrate                 # Run migrations
php artisan migrate:fresh --seed    # Reset DB + seed data
```

```
php artisan route:list              # Show all routes
php artisan tinker                  # Interactive REPL
php artisan test                    # Run PHPUnit tests
php artisan queue:work              # Process queued jobs
php artisan schedule:work           # Run scheduler locally
```

Tip: Use php artisan make: with flags to scaffold multiple files at once.

7. Project Folder Structure

```
taskforge/
|-- app/
|   |-- Http/
|   |   |-- Controllers/     # Handle HTTP requests
|   |   |-- Middleware/      # Filter requests (auth, etc.)
|   |   `-- Requests/        # Form validation classes
|   |-- Models/              # Eloquent models (database entities)
|   |-- Policies/            # Authorization rules
|   |-- Jobs/                # Queued background tasks
|   |-- Events/              # Domain events
|   |-- Listeners/           # React to events
|   `-- Providers/           # Service registration & bootstrapping
|-- bootstrap/
|   `-- app.php              # Application bootstrap (middleware, routes)
|-- config/                  # Configuration files
|-- database/
|   |-- migrations/          # Database schema versions
|   |-- seeders/             # Test/demo data
|   `-- factories/           # Fake data generators
|-- public/                  # Web root (index.php)
|-- resources/
|   |-- views/               # Blade templates
|   |-- css/                 # Source stylesheets
|   `-- js/                  # Source JavaScript
|-- routes/
|   |-- web.php              # Web routes (sessions, CSRF)
|   |-- api.php              # API routes (stateless, tokens)
|   `-- console.php          # Artisan commands & scheduler
|-- storage/                 # Logs, cache, uploads
|-- tests/                   # Automated tests
`-- vendor/                  # Composer packages (don't edit)
```

8. The Request Lifecycle (Preview)

```
HTTP Request
-> public/index.php
-> bootstrap/app.php
-> HTTP Kernel (middleware pipeline)
-> Router (match route)
-> Controller action
-> Response (view, JSON, redirect)
-> HTTP Response

Module 02 covers this in depth.
```

9. Hands-On: Set Up TaskForge

```
cd taskforge
copy .env.example .env
php artisan key:generate
```

```
# Create SQLite database file
echo. > database\database.sqlite

# Run migrations and seed demo data
php artisan migrate --seed

# Start server
php artisan serve
```

Login credentials after seeding:
- **Admin:** admin@taskforge.test / password
- **Member:** member@taskforge.test / password

10. Exercises

1. Change APP_NAME in .env to your name and confirm it appears in the browser tab.
2. Run php artisan route:list and identify the /up health route.
3. Open php artisan tinker and run App\Models\User::count().
4. Create a new route in routes/web.php that returns your name as JSON.
5. Switch DB_CONNECTION to MySQL (if available) and re-run migrations.

11. Self-Check Quiz

1. What is the purpose of APP_KEY?
2. Why should .env never be committed to Git?
3. What command generates a new migration file?
4. What folder is the web server's document root?
5. What is the difference between migrate and migrate:fresh?

Answers

1. Encrypts sessions, cookies, and other sensitive data. 2. It contains secrets (DB passwords, API keys). 3. `php artisan make:migration create_table_name` 4. `public/` - only this folder should be web-accessible. 5. `migrate` runs pending migrations; `migrate:fresh` drops all tables and re-runs everything.

Next Module

-> Module 01: PHP Fundamentals

Module 01: PHP Fundamentals for Laravel

Duration: 5-7 days | **Level:** Beginner | **Prerequisites:** Module 00

Learning Objectives

- Write modern PHP 8.2+ code used throughout Laravel
- Understand OOP concepts: classes, inheritance, interfaces, traits
- Use namespaces and autoloading
- Work with arrays, collections mindset, and type hints

1. Modern PHP Syntax

Variables & Types

```php
<?php

declare(strict_types=1);  // Enforce type declarations

$name = 'Alice';           // string
$age = 30;                 // int
$price = 19.99;            // float
$active = true;            // bool
$items = ['a', 'b', 'c'];  // array
$nothing = null;

// PHP 8+ union types
function process(int|string $value): void
{
echo $value;
}
```

Null Safe & Match (PHP 8+)

```php
// Nullsafe operator
$country = $user?->address?->country;

// Match expression (better than switch)
$status = match ($task->priority) {
'low' => 'info',
'medium' => 'warning',
'high', 'urgent' => 'danger',
default => 'secondary',
};
```

Named Arguments

```php
Task::create(
title: 'Fix bug',
project_id: 1,
due_at: now()->addDays(3),
);
```

Constructor Property Promotion (PHP 8+)

```
Laravel uses this everywhere:

class TaskController
{
public function __construct(
private TaskService $taskService,
) {}
}
```

---

2. Object-Oriented Programming

Classes & Objects

```php
<?php

namespace App\Services;

class TaskService
{
public function __construct(
private TaskRepository $repository,
) {}
```

```
public function complete(Task $task): Task
{
$task->status = 'completed';
$task->completed_at = now();

return $this->repository->save($task);
}
}
```

Inheritance & Abstract Classes

```
abstract class BaseController
{
protected function success(string $message): array
{
return ['success' => true, 'message' => $message];
}
}

class TaskController extends BaseController
{
// ...
}
```

Interfaces

```
interface Notifiable
{
public function sendNotification(string $message): void;
}

class EmailNotifier implements Notifiable
{
public function sendNotification(string $message): void
{
Mail::to($this->user)->send(new GenericMail($message));
}
}
```

Traits (Horizontal Reuse)

Laravel models use traits heavily:

```
trait HasTasks
{
public function tasks(): HasMany
{
return $this->hasMany(Task::class);
}
}

class User extends Authenticatable
{
use HasFactory, Notifiable, HasTasks;
}
```

Enums (PHP 8.1+)

```
enum TaskStatus: string
{
case Pending = 'pending';
case InProgress = 'in_progress';
case Completed = 'completed';

public function label(): string
{
```

```
return match ($this) {
self::Pending => 'Pending',
self::InProgress => 'In Progress',
self::Completed => 'Completed',
};
}
}

// Usage in model
protected function casts(): array
{
return ['status' => TaskStatus::class];
}
```

3. Namespaces & Autoloading

```
<?php
// File: app/Models/Task.php

namespace App\Models;

use App\Enums\TaskStatus;
use Illuminate\Database\Eloquent\Model;

class Task extends Model
{
// ...
}

Composer PSR-4 autoloading maps App\ -> app/:

"autoload": {
"psr-4": {
"App\\": "app/",
"Database\\Factories\\": "database/factories/",
"Database\\Seeders\\": "database/seeders/"
}
}

After adding new classes: composer dump-autoload
```

4. Arrays & Laravel Collections

PHP arrays are used everywhere, but Laravel provides **Collections** - fluent, chainable array operations:

```
$tasks = Task::all();

$overdue = $tasks
->filter(fn ($task) => $task->due_at?->isPast())
->sortBy('due_at')
->map(fn ($task) => $task->title)
->values();

Common Collection methods: map, filter, reduce, pluck, groupBy, first, each.
```

5. Error Handling

```
try {
$task = Task::findOrFail($id);
} catch (ModelNotFoundException $e) {
abort(404, 'Task not found');
```

```
} catch (Throwable $e) {
report($e);  // Log the error
throw $e;
}

Laravel's abort() helper throws HTTP exceptions:

abort(403, 'Unauthorized');
abort_if($user->cannot('update', $task), 403);
```

6. Closures & Arrow Functions

```
// Closure
$callback = function (Task $task) use ($user) {
return $task->user_id === $user->id;
};

// Arrow function (short, single expression)
$mine = $tasks->filter(fn ($t) => $t->user_id === $user->id);

Laravel routes use closures for simple endpoints:

Route::get('/health', fn () => response()->json(['ok' => true]));
```

7. Reading PHP in Laravel Codebase

When reading Laravel source, expect:

Pattern	Example
Facades	Cache::get('key') - static proxy to service
Dependency injection	Constructor/method type-hinted params
Magic methods	__get, __call on Eloquent models
Attributes	#[ObservedBy([TaskObserver::class])]
Return type declarations	: JsonResponse, : void

8. Hands-On Exercises

1. Create `app/Enums/TaskPriority.php` with cases: `Low`, `Medium`, `High`, `Urgent`.
2. Add a `label()` method returning human-readable strings.
3. In Tinker: `TaskPriority::High->label()` should return `"High"`.
4. Write a function `countByStatus(Collection $tasks): array` that returns `['pending' => 5, ...]`.

9. Self-Check Quiz

1. What does `declare(strict_types=1)` do?
2. Difference between `implements` and `extends`?
3. What is a trait used for?
4. How does PSR-4 autoloading find `App\Models\Task`?
5. What is the nullsafe operator?

Next Module

-> Module 02: Laravel Basics & Request Lifecycle

Module 02: Laravel Basics & Request Lifecycle

Duration: 4-5 days | **Level:** Beginner

The Request Lifecycle

Every HTTP request in Laravel follows this path:

```
1. public/index.php           -> Entry point
2. bootstrap/app.php          -> Creates Application instance
3. HTTP Kernel                -> Middleware pipeline
4. Router                     -> Matches URI to route
5. Controller / Closure       -> Business logic
6. Response                   -> View, JSON, redirect, file
7. Middleware (outbound)      -> Modify response
8. Send to browser
```

Middleware Pipeline

Middleware wraps your application like layers of an onion:

```
// bootstrap/app.php
->withMiddleware(function (Middleware $middleware): void {
$middleware->alias([
'admin' => \App\Http\Middleware\EnsureUserIsAdmin::class,
]);
})
```

Global middleware runs on every request. **Route middleware** runs only on assigned routes (`auth`, `guest`, `throttle`).

Service Container & Dependency Injection

Laravel's **service container** resolves class dependencies automatically:

```
// Laravel auto-injects DashboardService
public function __construct(private DashboardService $dashboardService) {}
```

Register bindings in `AppServiceProvider`:

```
$this->app->singleton(DashboardService::class, function ($app) {
return new DashboardService();
});
```

Configuration

File	Purpose
config/app.php	App name, timezone, locale
config/database.php	DB connections
config/cache.php	Cache drivers
config/queue.php	Queue drivers
config/mail.php	Mail settings

Access config: `config('app.name')` or `Config::get('app.name')`.

Environment overrides via `.env`: `APP_NAME=TaskForge`.

Facades

Facades provide static syntax for services:

```
Cache::get('key');          // Same as app('cache')->get('key')
Route::get(...);
Auth::user();
```

Service Providers

`bootstrap/providers.php` lists providers. `AppServiceProvider::boot()` is where you register policies, events, and view composers.

TaskForge example - `app/Providers/AppServiceProvider.php`:
- Registers `ProjectPolicy` and `TaskPolicy`
- Maps `TaskCompleted` event to `LogTaskCompletion` listener

Hands-On

1. Run `php artisan route:list` and trace one route through controller -> view.
2. Add a custom middleware that logs every request URL.
3. Use Tinker: `app()->make(DashboardService::class)`.

Next Module

-> Module 03: Routing & Controllers

Module 03: Routing & Controllers

Duration: 4-5 days | **Level:** Beginner-Intermediate

Defining Routes

Basic Routes (`routes/web.php`)

```
Route::get('/dashboard', [DashboardController::class,
'index'])->name('dashboard');

Route::middleware('auth')->group(function () {
Route::resource('projects', ProjectController::class);
Route::resource('tasks', TaskController::class);
Route::patch('/tasks/{task}/complete', [TaskController::class, 'complete'])
->name('tasks.complete');
});
```

Route Parameters

```
Route::get('/tasks/{task}', [TaskController::class, 'show']);
// {task} auto-resolves to Task model via Route Model Binding
```

Named Routes

Always name routes - use `route('tasks.show', $task)` in Blade instead of hardcoded URLs.

Resource Controllers

```
php artisan make:controller ProjectController --resource
```

Method	URI	Action	Route Name
GET	/projects	index	projects.index
GET	/projects/create	create	projects.create
POST	/projects	store	projects.store
GET	/projects/{project}	show	projects.show
GET	/projects/{project}/edit	edit	projects.edit
PUT/PATCH	/projects/{project}	update	projects.update
DELETE	/projects/{project}	destroy	projects.destroy

Controllers in TaskForge

Study these files:

- `app/Http/Controllers/ProjectController.php` - full CRUD with authorization
- `app/Http/Controllers/TaskController.php` - nested actions (comments, attachments, complete)
- `app/Http/Controllers/AuthController.php` - session auth

Controller Best Practices

1. **Thin controllers** - delegate complex logic to services (`DashboardService`)
2. **Type-hint dependencies** in constructor
3. **Use Form Requests** for validation (not inline in controller)
4. **Return explicit types**: `View`, `RedirectResponse`, `JsonResponse`

Route Model Binding

Laravel automatically injects models:

```
public function show(Task $task): View
{
// $task is already loaded from DB by ID in URL
$this->authorize('view', $task);
return view('tasks.show', compact('task'));
}
```

Custom binding in `AppServiceProvider`:

```
Route::bind('task', fn ($value) => Task::where('slug', $value)->firstOrFail());
```

API Routes (`routes/api.php`)

```
Route::middleware('auth:sanctum')->group(function () {
Route::apiResource('tasks', TaskApiController::class)->names('api.tasks');
});
```

API routes are prefixed with `/api` and use token auth (Sanctum).

Exercises

1. Add a route `GET /projects/{project}/archive` that sets `is_archived = true`.

2. Create `ProjectController@archive` with proper authorization.

3. List all TaskForge routes: `php artisan route:list --name=tasks`.

Next Module

-> Module 04: Blade Templates

Module 04: Blade Templates & Views

Duration: 4-5 days | **Level:** Beginner-Intermediate

Blade Basics

Blade compiles to plain PHP. Files live in `resources/views/`.

```
{{-- Output escaped HTML --}}
<h1>{{ $task->title }}</h1>

{{-- Unescaped (use carefully) --}}
{!! $htmlContent !!}

{{-- Directives --}}
@if($task->isOverdue())
<span class="text-red-600">Overdue</span>
@endif

@foreach($tasks as $task)
<li>{{ $task->title }}</li>
@endforeach

@forelse($tasks as $task)
<li>{{ $task->title }}</li>
@empty
<p>No tasks.</p>
@endforelse
```

Layouts & Sections

`resources/views/layouts/app.blade.php` - master layout:

```
<title>@yield('title') - {{ config('app.name') }}</title>
@yield('content')
```

Child view:

```
@extends('layouts.app')

@section('title', 'Dashboard')

@section('content')
<h1>Dashboard</h1>
@endsection
```

Components & Partials

Partial include - `resources/views/tasks/_form.blade.php`:

```
@include('tasks._form', ['action' => route('tasks.store'), 'method' => 'POST'])
```

Blade components (Laravel 7+):

```
php artisan make:component TaskCard
```

```
<x-task-card :task="$task" />
```

Forms & CSRF

Every POST form needs CSRF protection:

```
<form method="POST" action="{{ route('tasks.store') }}">
@csrf
@method('PUT')  {{-- For updates --}}
...
</form>
```

Validation Errors in Views

```
@if($errors->any())
<ul>
@foreach($errors->all() as $error)
<li>{{ $error }}</li>
@endforeach
</ul>
@endif

<input value="{{ old('title', $task->title ?? '') }}">
```

TaskForge Views to Study

File	Concepts
layouts/app.blade.php	Layout, nav, flash messages
dashboard.blade.php	Stats cards, loops, conditionals
tasks/_form.blade.php	Shared partial, old() helper
tasks/show.blade.php	Nested data, multiple forms
tasks/index.blade.php	Pagination, filters

Pagination

Controller:

```
$tasks = Task::paginate(15);
```

View:

```
{{ $tasks->links() }}
```

Exercises

1. Add a `@section('sidebar')` to the layout for project filters.
2. Create a Blade component `<x-status-badge :status="$task->status" />`.
3. Add flash message display for `session('error')`.

Next Module

-> Module 05: Database & Eloquent

Module 05: Database, Migrations & Eloquent ORM

Duration: 7-10 days | **Level:** Beginner-Intermediate

Migrations

Migrations are version-controlled database schema changes.

```
php artisan make:migration create_tasks_table
php artisan migrate
php artisan migrate:rollback
php artisan migrate:fresh --seed
```

TaskForge Migration Example

```
Schema::create('tasks', function (Blueprint $table) {
$table->id();
$table->foreignId('project_id')->constrained()->cascadeOnDelete();
$table->foreignId('user_id')->constrained()->cascadeOnDelete();
$table->string('title');
$table->text('description')->nullable();
$table->string('status')->default('pending');
$table->string('priority')->default('medium');
$table->timestamp('due_at')->nullable();
$table->timestamps();

$table->index(['status', 'due_at']);
});
```

Column Types Reference

Method	SQL Type
$table->string('name')	VARCHAR(255)
$table->text('body')	TEXT
$table->integer('count')	INT
$table->boolean('active')	BOOLEAN
$table->json('meta')	JSON
$table->foreignId('user_id')	UNSIGNED BIGINT
$table->timestamps()	created_at, updated_at

Eloquent Models

```
class Task extends Model
{
protected $fillable = ['title', 'project_id', 'status'];

protected function casts(): array
{
return [
'status' => TaskStatus::class,  // PHP Enum
'due_at' => 'datetime',
];
}
}
```

CRUD Operations

```
// Create
Task::create(['title' => 'Fix bug', 'project_id' => 1]);

// Read
$task = Task::find(1);
$task = Task::findOrFail(1);  // 404 if missing

// Update
$task->update(['status' => 'completed']);

// Delete
$task->delete();
```

Relationships

One-to-Many

```
// Project has many Tasks
class Project extends Model {
public function tasks(): HasMany {
return $this->hasMany(Task::class);
}
}

// Task belongs to Project
class Task extends Model {
public function project(): BelongsTo {
return $this->belongsTo(Project::class);
}
}
```

Many-to-Many (Assignees)

```
// Task.php
public function assignees(): BelongsToMany {
return $this->belongsToMany(User::class)->withTimestamps();
}

// Attach / sync
$task->assignees()->attach($userId);
$task->assignees()->sync([1, 2, 3]);
```

Eager Loading (N+1 Prevention)

```
// BAD: N+1 queries
$tasks = Task::all();
foreach ($tasks as $task) {
echo $task->project->name;  // Query per task!
}

// GOOD: 2 queries total
$tasks = Task::with(['project', 'assignees'])->get();
```

Query Scopes

```
// Model
public function scopeOverdue($query) {
return $query->where('due_at', '<', now())
```

```
->whereNotIn('status', ['completed', 'cancelled']);
}

// Usage
Task::overdue()->count();
```

Factories & Seeders

Factory - generate fake data:

```
Task::factory()->count(10)->create();
Task::factory()->overdue()->create();
```

Seeder - populate database:

```
php artisan db:seed
php artisan migrate:fresh --seed
```

TaskForge seeder creates admin, member, 2 projects, 6 tasks with comments.

Exercises

1. Add a tags table with many-to-many relationship to tasks.
2. Write a scope scopeDueThisWeek($query).
3. Use Tinker to query all overdue tasks with projects loaded.

Next Module

-> Module 06: Authentication & Authorization

Module 06: Authentication & Authorization

Duration: 5-7 days | **Level:** Intermediate

Session Authentication (TaskForge)

TaskForge implements session-based auth in AuthController:

```
// Login
if (Auth::attempt($credentials, $request->boolean('remember'))) {
$request->session()->regenerate();
return redirect()->intended(route('dashboard'));
}

// Logout
Auth::logout();
$request->session()->invalidate();
$request->session()->regenerateToken();
```

Route Protection

```
Route::middleware('guest')->group(function () {
Route::get('/login', ...);
});

Route::middleware('auth')->group(function () {
Route::get('/dashboard', ...);
});
```

Blade Auth Directives

```
@auth
<p>Welcome, {{ auth()->user()->name }}</p>
@endauth

@guest
<a href="{{ route('login') }}">Login</a>
@endguest
```

Laravel Breeze / Jetstream (Alternative)

For production apps, install scaffolding:

```
composer require laravel/breeze --dev
php artisan breeze:install blade
npm install && npm run build
php artisan migrate
```

Breeze provides login, register, password reset, and email verification.

Authorization with Policies

Policy - `app/Policies/TaskPolicy.php`:

```
public function update(User $user, Task $task): bool
{
return $user->id === $task->user_id
|| $task->project->user_id === $user->id
|| $user->isAdmin();
}
```

Usage in controller:

```
$this->authorize('update', $task);
// or
abort_unless($user->can('update', $task), 403);
```

Register in AppServiceProvider:

```
Gate::policy(Task::class, TaskPolicy::class);
```

Roles (TaskForge)

```
enum UserRole: string {
case Admin = 'admin';
case Member = 'member';
}

// User model
public function isAdmin(): bool {
return $this->role === UserRole::Admin;
}
```

For complex roles, consider spatie/laravel-permission.

API Authentication with Sanctum

```
// Create token
```

```
$token = $user->createToken('api-token')->plainTextToken;

// Protect routes
Route::middleware('auth:sanctum')->group(function () { ... });

// Test with Sanctum
Sanctum::actingAs($user);
```

Exercises

1. Add middleware EnsureUserIsAdmin for admin-only routes.
2. Prevent members from deleting projects they don't own.
3. Create an API token via Tinker and call GET /api/tasks.

Next Module

-> Module 07: REST API Development

Module 07: REST API Development

Duration: 5-7 days | **Level:** Intermediate

API Design Principles

Method	URI	Action
GET	/api/tasks	List tasks
POST	/api/tasks	Create task
GET	/api/tasks/{id}	Show task
PUT/PATCH	/api/tasks/{id}	Update task
DELETE	/api/tasks/{id}	Delete task

Use proper HTTP status codes: 200, 201, 204, 401, 403, 404, 422.

API Resources

Transform models to consistent JSON:

```
class TaskResource extends JsonResource
{
public function toArray(Request $request): array
{
return [
'id' => $this->id,
'title' => $this->title,
'status' => $this->status->value,
'project' => new ProjectResource($this->whenLoaded('project')),
];
}
}
```

Controller:

```
return TaskResource::collection($tasks);  // Paginated list
return new TaskResource($task);           // Single item
```

Sanctum Token Auth

```
composer require laravel/sanctum
php artisan vendor:publish --provider="Laravel\Sanctum\SanctumServiceProvider"
php artisan migrate
```

Generate token:

```
$token = $user->createToken('mobile-app')->plainTextToken;
// Returns: "1|abc123..."
```

API request:

```
curl -H "Authorization: Bearer 1|abc123..." \
-H "Accept: application/json" \
http://localhost:8000/api/tasks
```

API Versioning

```
// routes/api.php
Route::prefix('v1')->group(function () {
Route::apiResource('tasks', TaskApiController::class);
});
```

Rate Limiting

```
Route::middleware(['auth:sanctum', 'throttle:60,1'])->group(...);
```

TaskForge API Files

- `routes/api.php`
- `app/Http/Controllers/Api/TaskApiController.php`
- `app/Http/Resources/TaskResource.php`
- `tests/Feature/Api/TaskApiTest.php`

Exercises

1. Add `POST /api/tasks/{task}/complete` endpoint.
2. Create `CommentResource` and nest comments in task response.
3. Write API test for 403 when accessing another user's task.

Next Module

-> Module 08: Frontend Assets

Module 08: Frontend Assets (Vite, Tailwind, Alpine.js)

Duration: 3-5 days | **Level:** Intermediate

Vite Integration

Laravel uses Vite for bundling CSS/JS:

```
@vite(['resources/css/app.css', 'resources/js/app.js'])
```

Commands

```
npm install
npm run dev      # Development with HMR
npm run build    # Production build
```

File Structure

```
resources/
|-- css/app.css       # Tailwind imports
|-- js/app.js         # Entry point
`-- views/            # Blade templates
```

Tailwind CSS

Laravel 12 ships with Tailwind 4. Style in Blade:

```
<div class="bg-white p-6 rounded-lg shadow">
<h2 class="text-lg font-semibold text-gray-900">Tasks</h2>
</div>
```

Utility-first approach - no custom CSS files needed for most UI.

Alpine.js (Optional)

Lightweight JS for interactivity:

```
<div x-data="{ open: false }">
<button @click="open = !open">Toggle</button>
<div x-show="open">Content</div>
</div>
```

Livewire & Inertia (Advanced UI)

Tool	Use Case
Livewire	Full-stack components without writing API
Inertia.js	SPA feel with Vue/React + Laravel backend

```
composer require livewire/livewire
# or
composer require inertiajs/inertia-laravel
```

Production Assets

```
npm run build
```

```
Compiled files go to public/build/. Never run npm run dev in production.
```

Exercises

1. Run `npm run dev` and add a hover effect to task cards.
2. Add Alpine.js dropdown for task status filter.
3. Build assets and verify `@vite` works without dev server.

Next Module

-> Module 09: Validation & Form Requests

Module 09: Validation & Form Requests

Duration: 3-4 days | **Level:** Intermediate

Inline Validation

```
$validated = $request->validate([
'title' => ['required', 'string', 'max:255'],
'email' => ['required', 'email', 'unique:users'],
'due_at' => ['nullable', 'date', 'after_or_equal:today'],
]);
```

Form Request Classes

```
php artisan make:request StoreTaskRequest

class StoreTaskRequest extends FormRequest
{
public function authorize(): bool
{
return $this->user()->can('create', Task::class);
}

public function rules(): array
{
return [
'title' => ['required', 'string', 'max:255'],
'priority' => ['nullable', Rule::enum(TaskPriority::class)],
];
}

public function messages(): array
{
return [
'title.required' => 'Please enter a task title.',
];
}
}
```

Controller:

```
public function store(StoreTaskRequest $request): RedirectResponse
{
$task = Task::create($request->validated());
}
```

Common Validation Rules

Rule	Description
required	Must be present
nullable	Can be null
email	Valid email
unique:users,email	Unique in table
exists:projects,id	Foreign key exists
confirmed	Matches password_confirmation
max:255	Max length
mimes:jpg,png,pdf	File types
Rule::enum(Status::class)	PHP Enum value

Custom Rules

```
php artisan make:rule ValidProjectOwner
```

TaskForge Examples

- `StoreProjectRequest.php`
- `StoreTaskRequest.php`
- `StoreCommentRequest.php`

Exercises

1. Add validation: task due date cannot be more than 1 year ahead.
2. Create `UpdateTaskRequest` with `sometimes` rules.
3. Add custom error messages for all TaskForge forms.

Next Module

-> Module 10: File Storage

Module 10: File Storage & Media

Duration: 3-4 days | **Level:** Intermediate

Filesystem Configuration

`config/filesystems.php` defines disks:

Disk	Driver	Use
local	Local filesystem	Private files
public	Local + symlink	Public uploads
s3	AWS S3	Cloud storage

Uploading Files (TaskForge)

```
$request->validate([
'file' => ['required', 'file', 'max:5120', 'mimes:pdf,jpg,png'],
]);

$path = $request->file('file')->store('attachments', 'public');

Attachment::create([
'filename' => $path,
'original_name' => $file->getClientOriginalName(),
'mime_type' => $file->getMimeType(),
'size' => $file->getSize(),
]);
```

Public Access

```
php artisan storage:link

Creates public/storage -> storage/app/public symlink.
```

URL: `Storage::disk('public')->url($path)`

Downloading & Deleting

```
Storage::disk('public')->download($attachment->filename,
$attachment->original_name);
Storage::disk('public')->delete($attachment->filename);

TaskForge auto-deletes files in Attachment::booted().
```

S3 Production Setup

```
FILESYSTEM_DISK=s3
AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=...
AWS_DEFAULT_REGION=us-east-1
AWS_BUCKET=my-bucket
```

Exercises

1. Add image thumbnail generation with Intervention Image.
2. Limit total attachments per task to 5.
3. Add download route with authorization check.

Next Module

-> Module 11: Queues, Jobs & Scheduling

Module 11: Queues, Jobs & Scheduling

Duration: 4-5 days | **Level:** Intermediate-Advanced

Why Queues?

Offload slow work (emails, API calls, image processing) to background workers so HTTP responses stay fast.

Creating Jobs

```
php artisan make:job SendTaskDueReminder

class SendTaskDueReminder implements ShouldQueue
{
use Queueable;

public function __construct(public Task $task) {}

public function handle(): void
{
Notification::send($recipients, new TaskDueReminder($this->task));
}
}
```

Dispatch:

```
SendTaskDueReminder::dispatch($task);
SendTaskDueReminder::dispatch($task)->delay(now()->addMinutes(5));
```

Queue Configuration

```
QUEUE_CONNECTION=database   # Good for learning
# QUEUE_CONNECTION=redis      # Production recommended

php artisan queue:table
php artisan migrate
php artisan queue:work
php artisan queue:work --tries=3 --timeout=60
```

Task Scheduler

`routes/console.php`:

```
Schedule::command('tasks:send-reminders')->dailyAt('08:00');
```

Cron entry (production):

```
* * * * * cd /var/www/taskforge && php artisan schedule:run >> /dev/null 2>&1
```

Local testing:

```
php artisan schedule:work
php artisan tasks:send-reminders
```

Failed Jobs

```
php artisan queue:failed
php artisan queue:retry all
```

TaskForge Files

- `app/Jobs/SendTaskDueReminder.php`
- `app/Console/Commands/SendDueReminders.php`

- `app/Notifications/TaskDueReminder.php` (also queued)

Exercises

1. Create `ProcessAttachment` job for virus scanning simulation.
2. Configure `QUEUE_CONNECTION=sync` vs `database` and compare behavior.
3. Add retry logic with `$tries = 3` and `$backoff = [10, 60, 300]`.

Next Module

-> Module 12: Events & Listeners

Module 12: Events, Listeners & Notifications

Duration: 4-5 days | **Level:** Intermediate-Advanced

Events & Listeners

Decouple actions from reactions:

```
// Event
class TaskCompleted
{
use Dispatchable, SerializesModels;
public function __construct(public Task $task, public User $completedBy) {}
}

// Listener
class LogTaskCompletion
{
public function handle(TaskCompleted $event): void
{
ActivityLog::record('task.completed', $event->task, $event->completedBy);
}
}

// Dispatch
event(new TaskCompleted($task, $user));
```

Register in `AppServiceProvider`:

```
Event::listen(TaskCompleted::class, LogTaskCompletion::class);
```

Model Observers

```
php artisan make:observer TaskObserver --model=Task

class TaskObserver
{
public function updated(Task $task): void
{
if ($task->wasChanged('status')) {
// React to status change
}
}
}
```

Notifications

Multi-channel alerts (mail, database, Slack, SMS):

```
class TaskDueReminder extends Notification implements ShouldQueue
{
public function via(object $notifiable): array
{
return ['mail', 'database'];
}

public function toMail(object $notifiable): MailMessage { ... }
public function toArray(object $notifiable): array { ... }
}
```

Send:

```
$user->notify(new TaskDueReminder($task));
Notification::send($users, new TaskDueReminder($task));
```

Broadcasting (Real-time)

For WebSockets with Laravel Echo + Pusher/Ably:

```
composer require pusher/pusher-php-server
php artisan install:broadcasting
```

Exercises

1. Create `TaskAssigned` event that emails the assignee.
2. Add database notifications table: `php artisan notifications:table`.
3. Display unread notifications in the TaskForge navbar.

Next Module

-> Module 13: Caching & Performance

Module 13: Caching & Performance

Duration: 3-4 days | **Level:** Advanced

Cache Drivers

```
CACHE_STORE=database   # Learning
# CACHE_STORE=redis    # Production

php artisan cache:table && php artisan migrate
```

Basic Cache Operations

```
Cache::put('key', 'value', now()->addMinutes(10));
$value = Cache::get('key', 'default');
Cache::forget('key');
Cache::flush();
```

Remember Pattern (TaskForge DashboardService)

```
return Cache::remember(
"dashboard.stats.{$user->id}",
now()->addMinutes(5),
fn () => $this->computeStats($user),
);
```

Invalidate when data changes:

```
Cache::forget("dashboard.stats.{$user->id}");
```

Query Optimization

1. **Eager loading:** `with(['project', 'assignees'])`
2. **Select only needed columns:** `select(['id', 'title'])`
3. **Chunk large datasets:** `Task::chunk(100, fn ($tasks) => ...)`
4. **Database indexes** on frequently queried columns

Laravel Telescope (Debugging)

```
composer require laravel/telescope --dev
php artisan telescope:install
```

Monitors queries, requests, jobs, cache, mail.

Production Performance Checklist

- [ ] `php artisan config:cache`
- [ ] `php artisan route:cache`
- [ ] `php artisan view:cache`
- [ ] `php artisan event:cache`
- [ ] `composer install --optimize-autoloader --no-dev`
- [ ] Redis for cache, sessions, queues
- [ ] OPcache enabled on PHP

Exercises

1. Cache project list for 10 minutes per user.
2. Use `Cache::tags()` (Redis only) for grouped invalidation.
3. Profile a page with Telescope and fix one N+1 query.

Next Module

-> Module 14: Testing

Module 14: Testing (Unit, Feature, Browser)

Duration: 7-10 days | **Level:** Intermediate-Advanced

PHPUnit Setup

Laravel includes PHPUnit. Run tests:

```
php artisan test
php artisan test --filter=TaskTest
php artisan test --coverage
```

Feature Tests

Test HTTP endpoints and full flows:

```
class TaskTest extends TestCase
{
use RefreshDatabase;

public function test_user_can_create_task(): void
{
$user = User::factory()->create();
$project = Project::factory()->create(['user_id' => $user->id]);

$this->actingAs($user)
->post(route('tasks.store'), [
'project_id' => $project->id,
'title' => 'New task',
])
->assertRedirect();

$this->assertDatabaseHas('tasks', ['title' => 'New task']);
}
}
```

API Tests

```
Sanctum::actingAs($user);

$this->getJson('/api/tasks')
->assertOk()
->assertJsonCount(3, 'data');
```

Unit Tests

Test isolated classes without HTTP:

```
class DashboardServiceTest extends TestCase
{
use RefreshDatabase;

public function test_stats_returns_correct_counts(): void
{
$service = app(DashboardService::class);
// ...
}
}
```

Factories in Tests

```
$user = User::factory()->create();
Task::factory()->count(5)->overdue()->create([
'user_id' => $user->id,
]);
```

Mocking

```
Mail::fake();
Notification::fake();
Queue::fake();
Event::fake();

// Assert
Mail::assertSent(WelcomeMail::class);
Queue::assertPushed(SendTaskDueReminder::class);
```

Browser Tests (Laravel Dusk)

```
composer require laravel/dusk --dev
php artisan dusk:install

$this->browse(function (Browser $browser) {
$browser->visit('/login')
->type('email', 'admin@taskforge.test')
->type('password', 'password')
->press('Sign In')
->assertPathIs('/dashboard');
});
```

TaskForge Test Suite

File	Tests
tests/Feature/AuthTest.php	Login, register, logout
tests/Feature/TaskTest.php	CRUD, complete, scopes
tests/Feature/Api/TaskApiTest.php	Sanctum API

Testing Best Practices

1. Use `RefreshDatabase` for isolation
2. One assertion concept per test
3. Descriptive test names: `test_guest_cannot_view_tasks`
4. Test happy path AND edge cases
5. Run tests in CI on every push

Exercises

1. Add test: member cannot delete admin's project.
2. Add test: completing task fires `TaskCompleted` event.
3. Achieve 80%+ coverage on controllers.

Next Module

-> Module 15: Security

Module 15: Security Hardening

Duration: 3-5 days | **Level:** Advanced

Built-in Laravel Security

Feature	Protection
CSRF tokens	**Cross-site request forgery**
Password hashing (bcrypt/argon)	**Plaintext passwords**
Eloquent parameter binding	**SQL injection**
Blade {{ }} escaping	**XSS**
Signed URLs	**URL tampering**
Rate limiting	**Brute force**

CSRF Protection

All POST/PUT/DELETE forms need `@csrf`. API routes using Sanctum tokens are exempt.

Mass Assignment Protection

```
protected $fillable = ['title', 'description'];  // Whitelist
// OR
protected $guarded = ['id', 'is_admin'];           // Blacklist
```

Never use `$guarded = []` in production.

Authorization

Always check permissions:

```
$this->authorize('update', $task);
```

Never trust `$request->user_id` from client input.

Environment Security

```
APP_DEBUG=false          # NEVER true in production
APP_ENV=production
```

- Never commit `.env`
- Rotate `APP_KEY` if compromised
- Use secrets manager (AWS Secrets Manager, Doppler)

HTTPS & Headers

Force HTTPS in `AppServiceProvider`:

```
if (app()->environment('production')) {
URL::forceScheme('https');
}
```

Security headers middleware (or nginx):

```
X-Frame-Options: SAMEORIGIN
X-Content-Type-Options: nosniff
```

```
Strict-Transport-Security: max-age=31536000
```

Input Validation

```
Validate ALL user input. Use Form Requests. Sanitize file uploads (type, size, scan).
```

Dependency Security

```
composer audit

Enable Dependabot on GitHub for automatic security PRs.
```

Exercises

1. Add rate limiting to login: throttle:5,1.
2. Audit TaskForge for mass assignment vulnerabilities.
3. Configure APP_DEBUG=false and verify stack traces are hidden.

Next Module

-> Module 16: Deployment & Hosting

Module 16: Deployment & Hosting

Duration: 5-7 days | **Level:** Advanced

Hosting Options

Provider	Best For	Price
Laravel Forge	Managed VPS deployment	$12+/mo
Laravel Vapor	Serverless (AWS Lambda)	Usage-based
DigitalOcean	VPS (Droplets)	$6+/mo
AWS EC2 / Lightsail	Full control	Varies
Shared hosting	Simple apps (cPanel)	$3+/mo
Railway / Render	Quick deploy from Git	Free tier

Server Requirements

- PHP 8.2+ with extensions (mbstring, openssl, pdo, tokenizer, xml, ctype, json, bcmath)
- Composer 2.x
- Nginx or Apache
- MySQL 8+ / PostgreSQL 15+ / SQLite (dev only)
- Redis (recommended for cache/queue)
- Node.js (build step only)

Deployment Checklist

1. Prepare Application

```
composer install --optimize-autoloader --no-dev
npm ci && npm run build
php artisan config:cache
php artisan route:cache
php artisan view:cache
php artisan event:cache
```

2. Environment

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://taskforge.example.com

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_DATABASE=taskforge
DB_USERNAME=taskforge
DB_PASSWORD=strong-password

CACHE_STORE=redis
QUEUE_CONNECTION=redis
SESSION_DRIVER=redis

MAIL_MAILER=smtp
MAIL_HOST=smtp.mailgun.org
```

3. Nginx Configuration

See `taskforge/deploy/nginx.conf`:

```
server {
listen 80;
server_name taskforge.example.com;
root /var/www/taskforge/public;

add_header X-Frame-Options "SAMEORIGIN";
add_header X-Content-Type-Options "nosniff";

index index.php;
charset utf-8;

location / {
try_files $uri $uri/ /index.php?$query_string;
}

location ~ \.php$ {
fastcgi_pass unix:/var/run/php/php8.2-fpm.sock;
fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
include fastcgi_params;
}

location ~ /\.(?!well-known).* {
deny all;
}
}
```

4. File Permissions

```
chown -R www-data:www-data /var/www/taskforge
chmod -R 755 /var/www/taskforge
chmod -R 775 storage bootstrap/cache
```

5. Run Migrations

```
php artisan migrate --force
php artisan storage:link
```

6. Process Managers

Queue worker (Supervisor):

```
[program:taskforge-worker]
process_name=%(program_name)s_%(process_num)02d
command=php /var/www/taskforge/artisan queue:work redis --sleep=3 --tries=3
--max-time=3600
autostart=true
autorestart=true
user=www-data
numprocs=2
```

Scheduler cron:

```
* * * * * www-data cd /var/www/taskforge && php artisan schedule:run >>
/dev/null 2>&1
```

SSL with Let's Encrypt

```
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d taskforge.example.com
```

Zero-Downtime Deployment

Use **Forge**, **Envoyer**, or **GitHub Actions** with:

1. Clone to new release directory
2. composer install --no-dev
3. php artisan migrate --force
4. Symlink current -> new release
5. Reload PHP-FPM
6. Restart queue workers: php artisan queue:restart

Docker / Laravel Sail

```
composer require laravel/sail --dev
php artisan sail:install
./vendor/bin/sail up
```

CI/CD with GitHub Actions

See taskforge/deploy/github-actions.yml for automated test + deploy pipeline.

Exercises

1. Deploy TaskForge to a free Render.com or Railway instance.
2. Set up SSL and verify https:// works.
3. Configure Supervisor for queue workers.

Next Module

-> Module 17: Maintenance & Monitoring

Module 17: Maintenance, Monitoring & Logging

Duration: 3-5 days | **Level:** Advanced

Logging

Laravel uses Monolog. Configure in `config/logging.php`:

```
LOG_CHANNEL=stack
LOG_LEVEL=debug   # production: error or warning
```

Write logs:

```
Log::info('Task created', ['task_id' => $task->id]);
Log::error('Payment failed', ['exception' => $e]);
report($exception);  // Log + optionally send to external service
```

Log channels: `single`, `daily`, `slack`, `papertrail`, `syslog`.

Error Tracking

Service	Integration
Sentry	sentry/sentry-laravel
Flare	spatie/laravel-ignition (included)
Bugsnag	bugsnag/bugsnag-laravel

```
composer require sentry/sentry-laravel
php artisan sentry:publish
```

Application Monitoring

- **Laravel Pulse** - real-time app metrics
- **Laravel Horizon** - Redis queue dashboard
- **New Relic / Datadog** - APM

```
composer require laravel/pulse
php artisan vendor:publish --provider="Laravel\Pulse\PulseServiceProvider"
```

Health Checks

```
Built-in: GET /up

Custom health route:

Route::get('/health', function () {
return response()->json([
'status' => 'ok',
'database' => DB::connection()->getPdo() ? 'connected' : 'failed',
'cache' => Cache::has('health_check') || Cache::put('health_check', true, 60),
]);
});
```

Maintenance Mode

```
php artisan down --secret="bypass-token"
# Site shows maintenance page; access via /bypass-token

php artisan up
```

With message:

```
php artisan down --render="errors::503" --retry=60
```

Database Maintenance

```
# Backup
php artisan db:backup  # with spatie/laravel-backup

# Manual MySQL dump
mysqldump -u user -p taskforge > backup.sql

# Optimize
php artisan model:prune  # Delete old soft-deleted records
```

Spatie Backup

```
composer require spatie/laravel-backup

Schedule daily backups to S3.
```

Performance Monitoring Checklist

- [ ] Set up error tracking (Sentry)
- [ ] Monitor queue depth and failed jobs
- [ ] Disk space alerts on server
- [ ] Database slow query log
- [ ] Uptime monitoring (UptimeRobot, Pingdom)
- [ ] SSL certificate expiry alerts

Log Rotation

Configure daily channel with days retention:

```
'daily' => [
'driver' => 'daily',
'path' => storage_path('logs/laravel.log'),
'level' => 'debug',
'days' => 14,
],
```

Exercises

1. Add Sentry or configure Slack log channel for errors.
2. Create custom Artisan command app:health-check for monitoring.
3. Set up daily database backup schedule.

Next Module

-> Module 18: Advanced Topics & Architecture

Module 18: Advanced Topics & Architecture

Duration: Ongoing | **Level:** Advanced

Service Layer Pattern

Keep controllers thin; business logic in services:

```
app/
|-- Services/
|   |-- TaskService.php
|   `-- DashboardService.php
|-- Repositories/          # Optional data access layer
|   `-- TaskRepository.php
`-- Actions/               # Single-purpose classes
`-- CompleteTask.php

class CompleteTask
{
public function execute(Task $task, User $user): Task
{
$task->markComplete();
event(new TaskCompleted($task, $user));
return $task;
}
}
```

Repository Pattern

Abstract database access:

```
interface TaskRepositoryInterface
{
public function findOverdue(): Collection;
}

class EloquentTaskRepository implements TaskRepositoryInterface { ... }
```

Bind in `AppServiceProvider`:

```
$this->app->bind(TaskRepositoryInterface::class, EloquentTaskRepository::class);
```

Multi-Tenancy

For SaaS apps serving multiple organizations:

- stancl/tenancy
- Database-per-tenant or shared database with `tenant_id`

Microservices vs Monolith

Laravel excels as a **modular monolith**:

```
app/
|-- Domains/
|   |-- Tasks/
|   |   |-- Models/
|   |   |-- Controllers/
|   |   `-- Services/
|   `-- Billing/
```

Extract to microservices only when you have clear scaling boundaries.

Laravel Packages Ecosystem

Package	Purpose
spatie/laravel-permission	Roles & permissions
spatie/laravel-medialibrary	Media management
spatie/laravel-activitylog	Audit logging
laravel/scout	Full-text search (Algolia/Meilisearch)
laravel/cashier	Stripe subscriptions
laravel/socialite	OAuth login
barryvdh/laravel-debugbar	Dev debugging

Design Patterns in Laravel

Pattern	Laravel Feature
Factory	Model factories
Observer	Model observers
Strategy	Driver-based systems (cache, mail)
Decorator	Middleware
Command	Artisan commands, Jobs
Repository	Custom implementations

API-First & Mobile

1. Build API with Sanctum/Passport
2. Use API Resources for consistent JSON
3. Version your API (`/api/v1/`)
4. Document with Scramble or OpenAPI

Continuing Your Journey

Build Portfolio Projects

1. **Blog CMS** - posts, categories, comments, admin panel
2. **E-commerce** - products, cart, Stripe checkout
3. **SaaS Dashboard** - subscriptions, teams, billing
4. **Real-time Chat** - broadcasting, presence channels

Certifications & Community

- Laravel certification (official)
- Laracasts learning paths
- Laravel Live conferences
- Contribute to open source

Stay Current

- Read Laravel News

- Follow release notes each major version
- Upgrade guide: `laravel.com/docs/upgrade`

Congratulations!

You've completed the TaskForge Laravel curriculum from setup through production. You now have:

- 19 learning modules (00-18)
- A full-featured example application
- Automated test suite
- Deployment configurations
- Real-world patterns used in professional Laravel development

Keep building. Keep testing. Keep shipping.

Curriculum Complete

Return to README for the full roadmap and progress checklist.